

TEW-KB

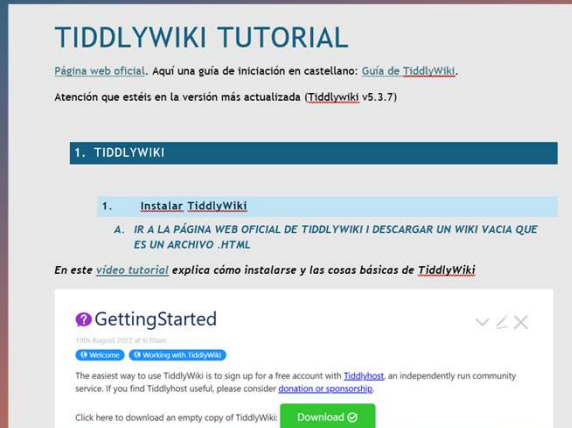
TiddlyWiki - TypeDB

TiddlyWiki



TiddlyWiki Documents

- TiddlyWiki Tutorial



- Folders

A screenshot of a folder list. The folders are listed with yellow folder icons to the left of their names. The folders are: "DecisionalProcess_Template", "EDIT_Template", "New_ELEMENT", "PILOT_CASES", and "VIEW_Template".

DecisionalProcess_Template

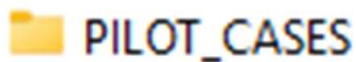
EDIT_Template

New_ELEMENT

PILOT_CASES

VIEW_Template

TEW-KG Projects



1. Pilot_Cases: projects in .html files

Name	Kind	Version
 saopedrotewkg		5.3.7
 caldesmontbuitewkg		5.3.7
 chaudesaiguestewkg		5.3.7
 templatetewkg		5.3.7
 templatepilotcases		5.3.7

+ Create  Create from template  Claim Tiddlyspot site  Download all

Name
 20251029_caldesmontbuitewkg.html
 20251029_chaudesaiguestewkg.html
 20251029_saopedrotewkg.html
 20251029_templatepilotcases.html
 20251029_templatetewkg.html
 Identification_TEW-KG_pilot_cases.docx

Before using TEW-KG on the Web Server, ensure the Pilot Case responsible has saved and closed the tool.

TiddlyWiki Scripts

 New_ELEMENT

2. Script: New Element in the Menu

Contents ▾

Search...

+ New Element

CreateNewElemen     

t

22nd October 2025 at 9:25am

1. Write an element's name:

- without spaces.
- each word should start with a capital letters
Ej. RiskSources

2. Choose an OBJECT type:

3. Choose a DOMAIN type:

3. Press the button

TiddlyWiki Scripts

EDIT_Template

VIEW_Template

DecisionalProcess_Template

3. Scripts: Create and View the attributes / decisional Process

MagicMountains



22nd September 2025 at 4:05pm

Nature

TerritorialLabel

TerritorialLabel:

labels to evaluate and recognize a territory's compliance with specific standards, criteria, or policies in areas such as sustainability, public management, human rights, or economic and environmental development.

Decisional Process:

Concerned Domain: tourism & leisure

Duration:

Document of Reference: <http://montanhasmagicas.pt/pt/>

Validation Process: Yes

Status Development: implemented

Comments: "to be expanded" triplet set: subject: MagicMoutains
predicate: promotes object: SustainableTourism

Decisional Process:

Concerned Domain:

Affected Time Period:

Document of Reference:

Validation Process:

TypeDB

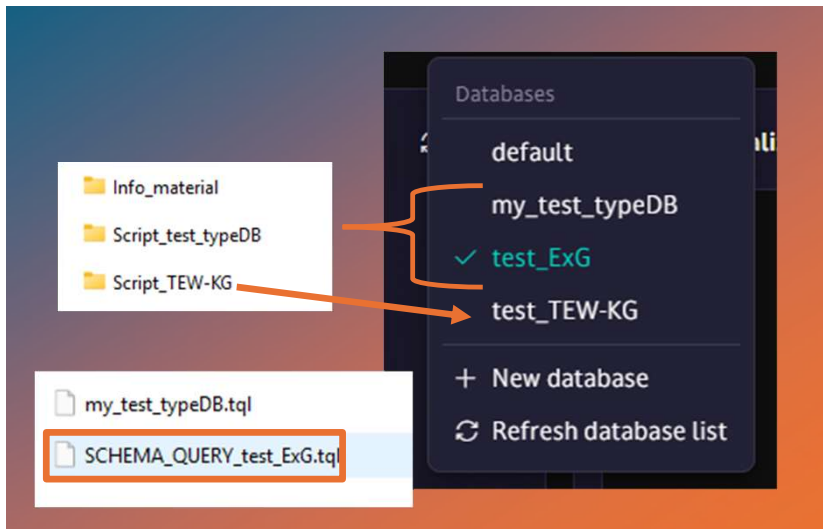
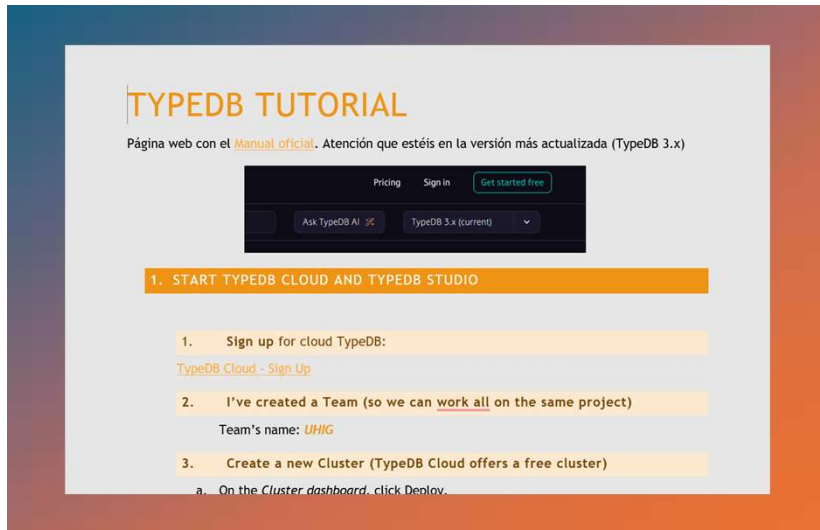


TypeDB Documents

❑ TypeDB Tutorial

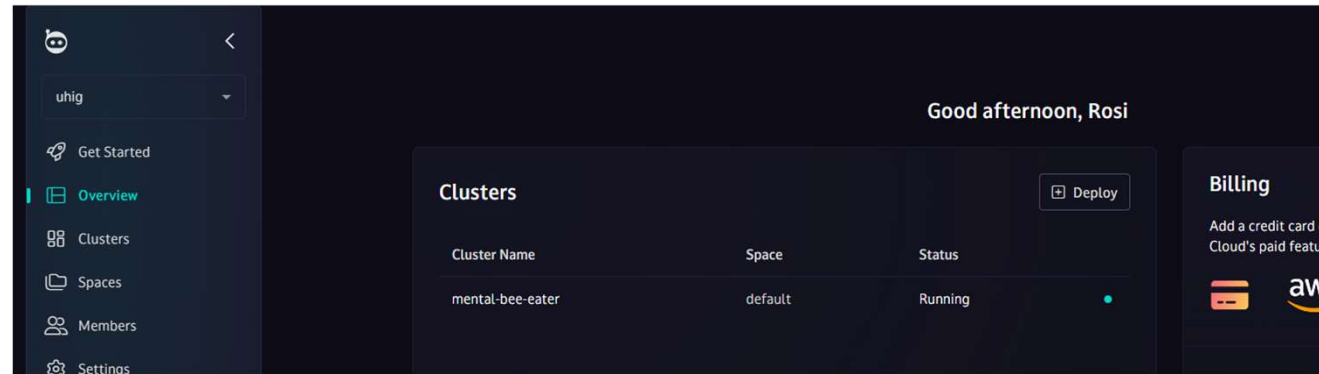
- Starting TypeDB Cloud & TypeDB Studio
- TypeDB Console Installation
- TypeDB Model Creation
 - ✓ Defining TypeDB SCHEMA
 - ✓ Querying the SCHEMA
 - ✓ Inserting Instances
 - ✓ Querying entities, relations and attributes.

❑ Folders



¿Cómo acceder a TypeDB?

TypeDB Studio & Cloud



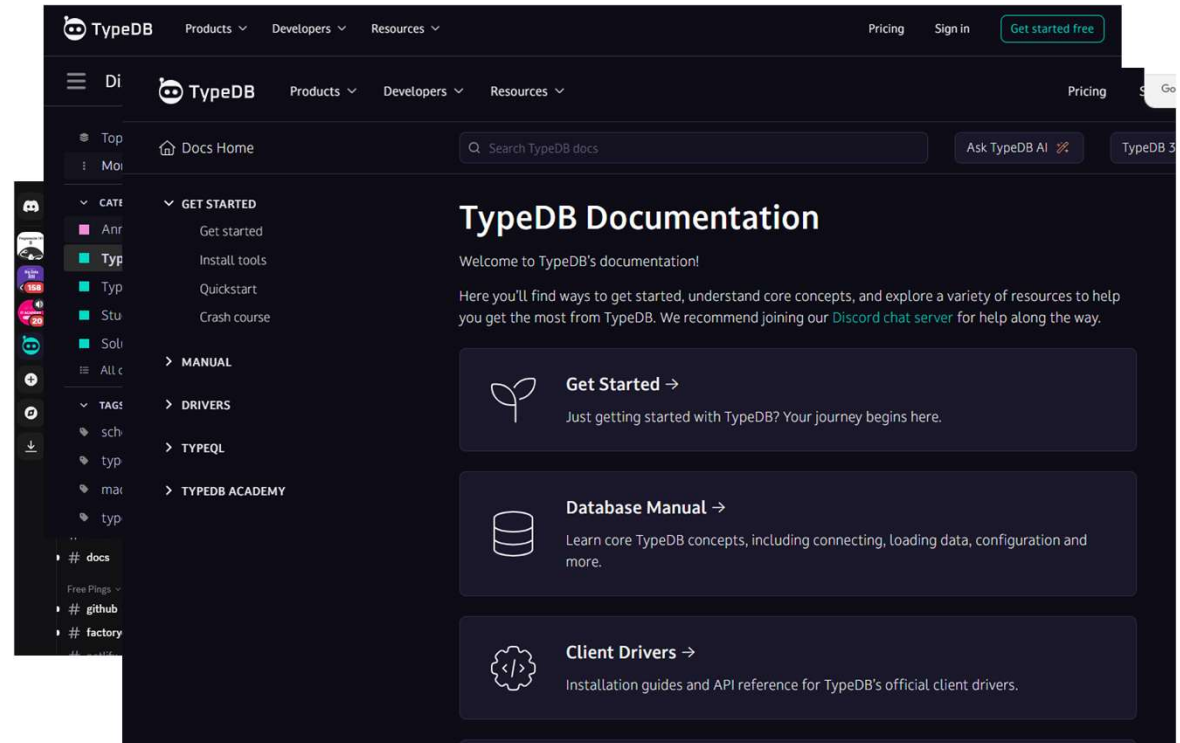
- To access the **TypeDB Studio** interface, you need to connect to **TypeDB Cloud** and create a server (“**cluster**”):
Instructions for connecting are provided in the document: *TypeDB_Tutorial.doc*
- You can use the “cluster” I have already set up for practice, **logging in as another member**.

Information & Formation Sources

Additional Resources

Check out the community or learn more about TypeDB.

- [typedb.com](#)
- [Chat Server](#)
- [Forum](#)
- [Docs](#)
- [TypeDB Manual](#)
- [TypeQL Guide](#)



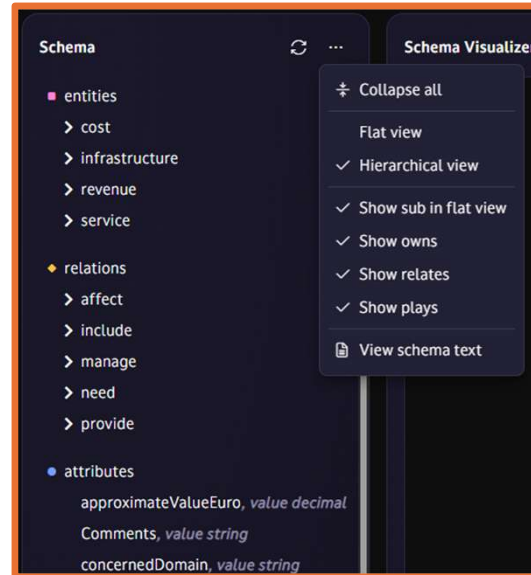
 Video tutorials and some documentation **are outdated**.

 Very active Discord channel with fast responses

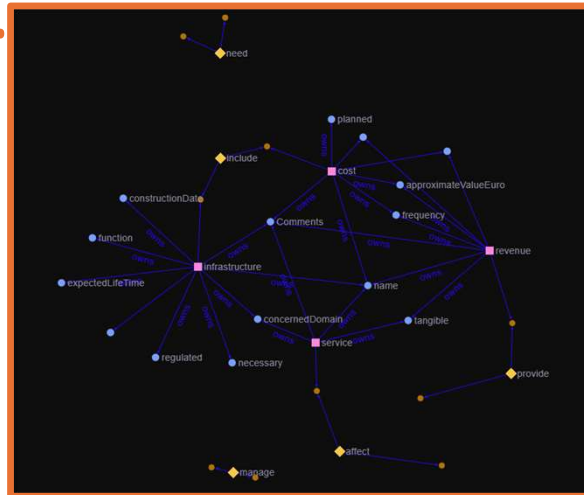
TypeDB Tutorial

Interface Updated:
SCHEMA

Schema dropdown



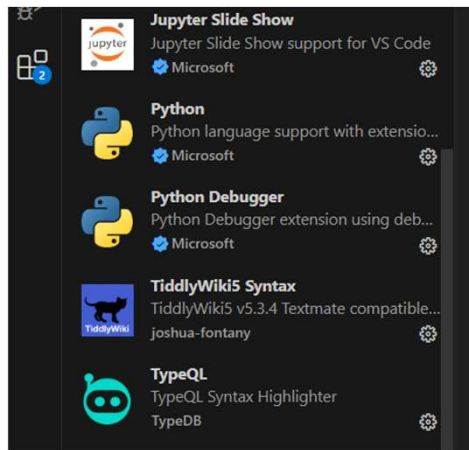
Schema Visualizer



Schema Script

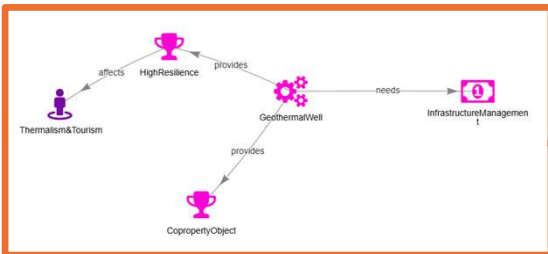
Full schema text [test_TEW-KG]

```
1 define
2
3 attribute name,
4 value string;
5 attribute approximateValueEuro,
6 value decimal;
7 attribute Comments,
8 value string;
9 attribute concernedDomain,
10 value string;
11 attribute constructionDate,
12 value string;
13 attribute estimationPrecision,
14 value decimal;
15 attribute expectedLifeTime,
16 value string;
17 attribute frequency,
18 value string;
19 attribute function,
20 value string;
21 attribute regulated,
22 value string;
23 attribute necessary,
24 value string;
25 attribute planned,
26 value string;
27 attribute referencePeriod
```



TEW-KG SCHEMA

TypeQL Extension for VSC



define

✓ **entity**

Infrastructure, Cost, Revenue, Service

✓ **attribute**

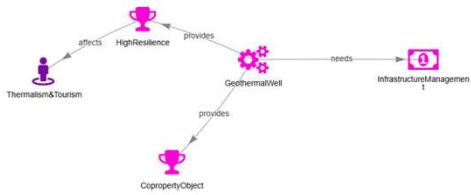
✓ **relation**

```
Script_TEW-KG > SCHEMA_test_TEW-KG.tql
1  ## SCHEMA TEW-KG
2  define
3
4  # definir atributos
5      attribute name, value string;
6      attribute approximateValueEuro, value decimal;
7      attribute Comments, value string;
8      attribute concernedDomain, value string;
9      attribute constructionDate, value string;
10     attribute estimationPrecision, value decimal;
11     attribute expectedLifeTime, value string;
12     attribute frequency, value string;
13     attribute function, value string;
14     attribute regulated, value string;
15     attribute necessary, value string;
16     attribute planned, value string;
17     attribute referencePeriod, value string;
18     attribute statusDevelopment, value string;
19     attribute tangible, value string;
20
21 # definir la entidad infraestructura
22 entity infraestructura,
23     owns name @key,
24     owns concernedDomain,
25     owns function,
26     owns regulated,
27     owns necessary,
```

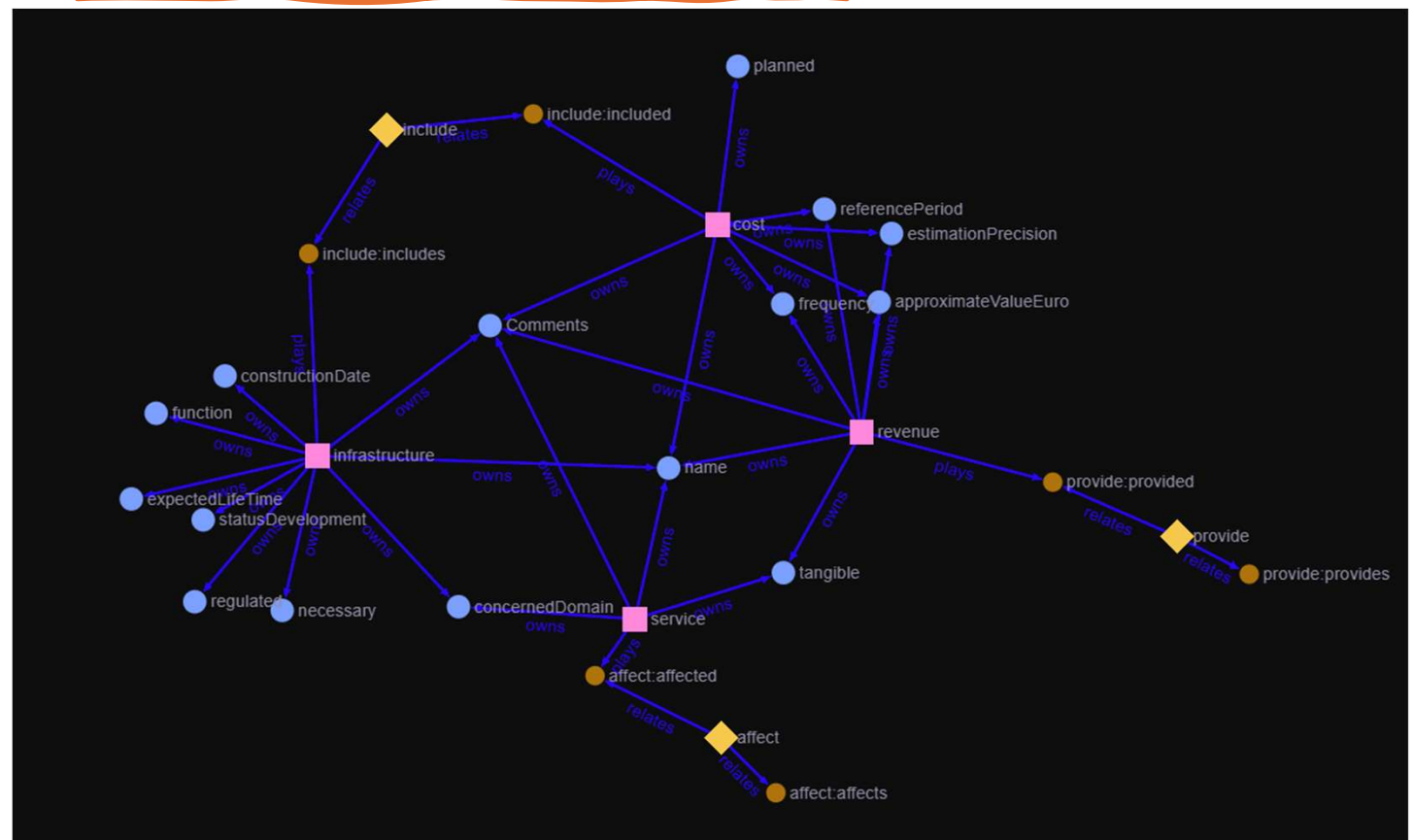
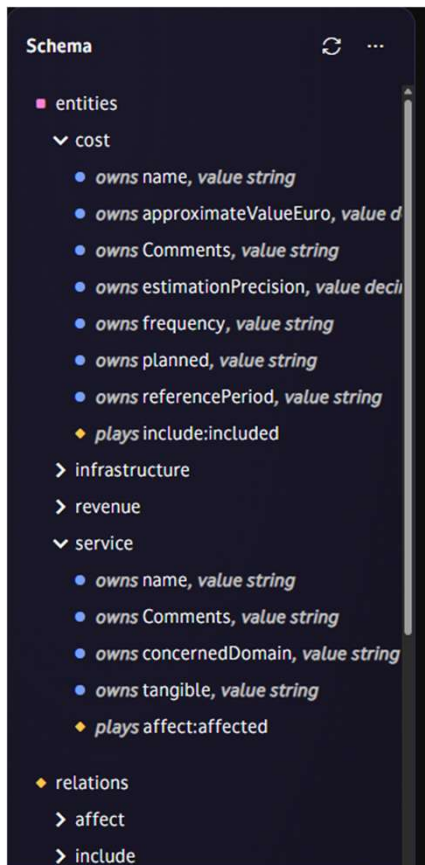
TEW-KG write auto

Query </> read write ✓ at ALPHA

```
1  define schema
2  entity infraestructura,
3      owns name @key,
4      owns concernedDomain,
5      owns function,
6      owns regulated,
7      owns necessary,
8      owns constructionDate,
9      owns expectedLifeTime,
10     owns statusDevelopment,
11     owns Comments,
12     plays include:includes;
```



TEW-KG SCHEMA



TEW-KG Instances

How to insert in
TypeDB?

```
# introducir nueva información en la base de datos como instancias
v insert
  $x isa costs, has quantity 4000;
  $w isa costs, has frequency "year";
  $y isa infrastructure, has name "Balneary heating";
  $y isa infrastructure, has status "working";

# insertar una relación entre instancias que ya están dentro de la base de datos
v match
  $u0 isa costs, has quantity 4000;
  $u1 isa infrastructure, has name "Balneary heating";

v insert
  product (produced: $u0, produces: $u1);
```

Query </> code 🔗 chat ALPHA

```
1 match
2   $x isa revenue;
3 fetch {
4   $x.*
5 };
```

Results log ✓ table graph raw

tangible	referencePeriod	name	estimationPrecision	frequency	Comments
"Yes"	"2025"	"HighResilience"	"40.0dec"	"as needed"	"The system consists of short, shallow pipelines serving near
"Yes"	"2025"	"CopropertyObject"	"40.0dec"	"annually"	"Invesments to fund the com

Query </> code 🔗 chat ALPHA

```
1
2 match
3   $x isa revenue;
4 fetch {
5   "name": $x.name,
6   "Frequency": $x.frequency,
7   "Approximate Value (Euros)": $x.approximateValueEuro,
8 };
```

Results log ✓ table graph raw

Approximate Value (Euros)	frequency	name
"100000.0dec"	"as needed"	"HighResilience"
"70000.0dec"	"annually"	"CopropertyObject"

Query </> code 🔗 chat ALPHA

```
1 match $x isa revenue, has approximateValueEuro > 80000;
2 limit 2;
3 fetch {
4   $x.*
5 };
```

referencePeriod ↑ name approximateValueEuro Comments

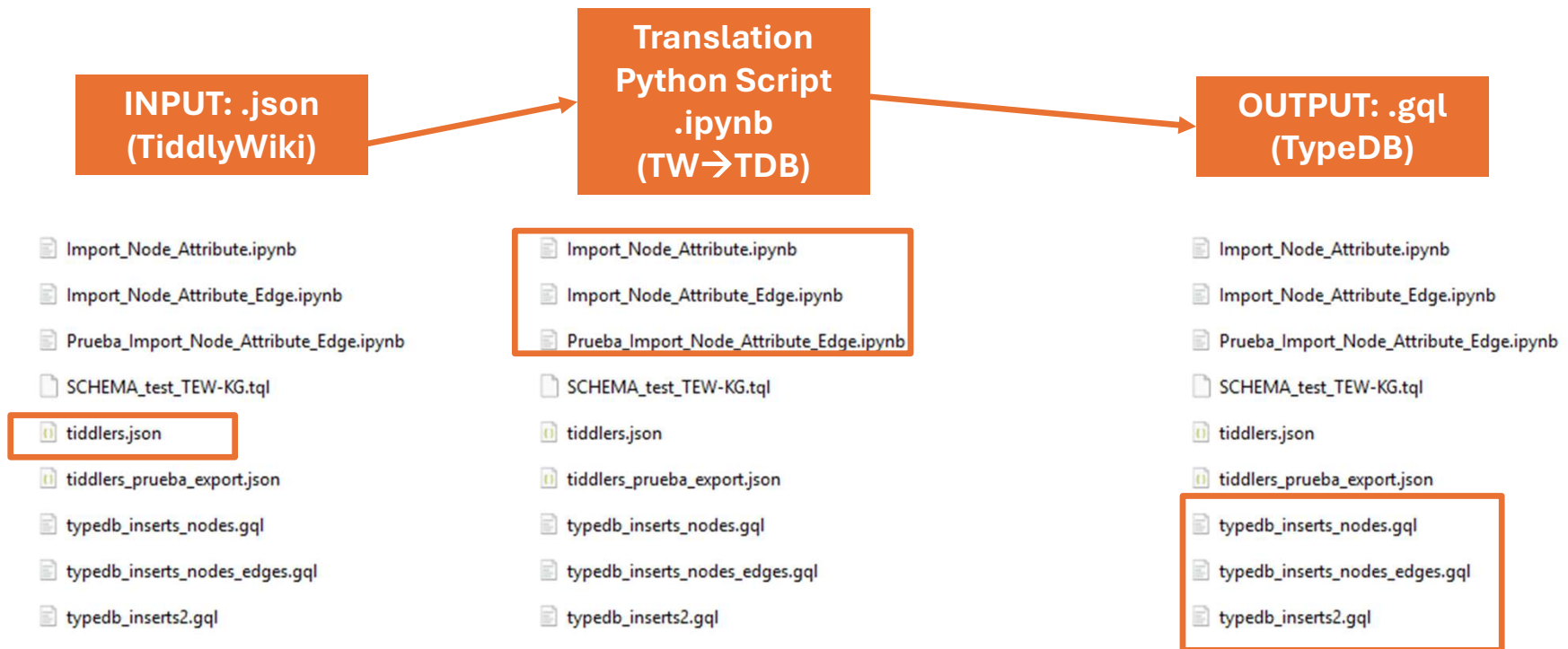
"2025"	"HighResilience"	"100000.0dec"	"The system consists of short, shallow pipelines serving near
--------	------------------	---------------	---

Querying Instances

TEW-KB

TiddlyWiki
→ TypeDB

TiddlyWiki → TypeDB



TiddlyWiki → TypeDB

1. Getting the INPUT FILE

The screenshot displays the TiddlyWiki interface with several key elements highlighted by orange boxes and arrows:

- Top Left:** A text box containing the URL `$/plugins/felixhayashi/tiddlymap/graph/views/CM_Infrastructure/filter/nodes`. An orange box highlights the `$/plugins/f` portion.
- Top Center:** The "General TEMPLATE" header and a search bar. Below the search bar, a dropdown menu is open, showing "CM_Infrastructure" selected. An orange box highlights this dropdown.
- Bottom Left:** The "Advanced Search" section. A search bar contains the text "CM_Infrastructure/filter/nodes". Below it, a list of matches is shown, with the first match highlighted by an orange box: `$/plugins/felixhayashi/tiddlymap/graph/views/CM_Infrastructure/filter/nodes`.
- Center:** A diagram illustrating the relationships between various entities. The entities are represented by icons: a person for "Thermalism&Tourism", a trophy for "HighResilience", a gear for "GeothermalWell", a trophy for "CoproperyObject", and a person for "InfrastructureManagement". The relationships are labeled: "affects" (from Thermalism&Tourism to HighResilience), "provides" (from HighResilience to GeothermalWell), "needs" (from GeothermalWell to InfrastructureManagement), and "provides" (from GeothermalWell to CoproperyObject).
- Bottom Right:** A table with two columns: "filter" and "title". The "filter" column contains a complex TypeDB query: `[field:tmap.id[353b8679-d8dc-4ae8-95f5-d5aa95ea30f2]] [field:tmap.id[782b22bd-13ba-4a69-8fbb-54d5af6c76d5]] [field:tmap.id[b7ab591c-c26d-4155-92cb-cad6a8e8870d]] [field:tmap.id[b76d71b7-6166-4406-8816-d61b0780668b]] [field:tmap.id[2500d4cb-a91e-41f5-8013-112449106fc8]]`. The "title" column contains the URL: `$/plugins/felixhayashi/tiddlymap/graph/views/CM_Infrastructure/filter/nodes`. An orange box highlights the entire table.

TiddlyWiki → TypeDB

1. Getting the INPUT FILE

\$/plugins/f
elixhayashi
/tiddlymap/graph/views/CM_I
nfrastructure/filter/nodes

Tools Backlinks Tagging List Listed Fields Advanced

filter	[field:tmap.id[353b8679-d8dc-4ae8-95f5-d5aa95ea30f2]] [field:tmap.id[782b22bd-13ba-4a69-8fbb-54d5af6c76d5]] [field:tmap.id[b7ab591c-c26d-4155-92cb-cad6a8e8870d]] [field:tmap.id[b76d71b7-6166-4406-8816-d61b0780668b]] [field:tmap.id[2500d4cb-a91e-41f5-8013-112449106fc8]]
title	\$/plugins/felixhayashi/tiddlymap/graph/views/CM_Infrastructure/filter/nodes

\$/AdvancedS
earch

Standard System Shadows Filter

Search via a [filter expression](#)

d[2500d4cb-a91e-41f5-8013-112449106fc8]]

5 matches

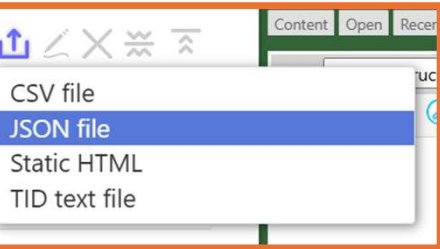
GeothermalWell
InfrastructureManagement
HighResilience
CopropertyObject
Thermalism&Tourism

CSV file
JSON file
Static HTML
TID text file

TiddlyWiki → TypeDB

Instances

INPUT: .json
(TiddlyWiki)



```
Import_Node_Attribute_Edge.ipynb • {} tiddlers.json X
C: > Users > rosamaria.moreno > OneDrive - Institut Cartogràfic i Geològic de Catalunya > Documents > Rosi > TypeDB > {} tiddlers.json > ...
1 [{"created":"20251009085223375","tags":"Infrastructure Heat Water","modified":"20251009085322911","title":"GeothermalWell","concernedDom...
```

```
Import_Node_Attribute_Edge.ipynb • {} tiddlers.json
C: > Users > rosamaria.moreno > OneDrive - Institut Cartogràfic i Geològic de Catalunya > Documents > Rosi > TypeDB > Import_
+ Code + Markdown | ▶ Run All ↺ Restart ≡ Clear All Outputs | Jupyter Variables ≡ Outline ...
▶
import json

# load the tiddlers JSON from the file
with open("tiddlers.json", "r", encoding="utf-8") as f:
    nodesAll = json.load(f) # convierte el texto (str) en diccionario de python
```

TiddlyWiki → TypeDB

```
# Crear el diccionario de todos los nodos
nodes_dict = {}

# Para cada elemento de la lista, asignar como nombre del nodo el elemento (atributo) "title"
for node in nodesAll:
    nomNode = node["title"]

    (variable) nodeAttributes: dict # Atributos para cada nodo (sin el elemento "title")
    nodeAttributes = {}
    # convertir los elementos de la lista en clave:valor de un diccionario
    for key, value in node.items():
        if key != "title":
            nodeAttributes[key] = value
            #print(key, value)
    # Asignar al nombre del nodo (nomNode) el subdiccionario de los atributos
    nodes_dict[nomNode] = nodeAttributes

print(nodes_dict)

# Crear lista de almacenamiento de instrucciones para insertar los nodos en TypeDB
typedb_statements = []

# Contadores por tipo de entidad
counters = {
    "cost": 1,
    "infrastructure": 1,
    "service": 1,
    "revenue": 1
}

# Create entities for each node
for node_id, node in nodes_dict.items():
    entityType = node.get("tags", "").split()[0].lower()
    approximateValueEuro = node.get("approximateValueEuro", "")
    Comments = node.get("Comments", "")
    concernedDomain = node.get("concernedDomain", "")
```

Instances

Translation
Python Script
.ipynb
(TW→TDB)

```
{'GeothermalWell': {'created': '20251009085223375', 'tags': 'Infrastructure Heat Water', 'mo
insert
$infra1 isa infrastructure, has name "GeothermalWell";
$infra1 isa infrastructure, has concernedDomain "water managment";
$infra1 isa infrastructure, has function "water capture process";
$infra1 isa infrastructure, has regulated "Yes";
$infra1 isa infrastructure, has necessary "Yes";
$infra1 isa infrastructure, has constructionDate "1997";
$infra1 isa infrastructure, has Comments "";

insert
$cost1 isa cost, has name "InfrastructureManagement";
$cost1 isa cost, has approximateValueEuro 100000;
$cost1 isa cost, has estimationPrecision 40;
$cost1 isa cost, has frequency "irregular";
$cost1 isa cost, has planned "Yes";
$cost1 isa cost, has referencePeriod "1990-2025";
$cost1 isa cost, has Comments "Complete system, including the well, 2 pumps, collect

insert
$rev1 isa revenue, has name "HighResilience";
$rev1 isa revenue, has approximateValueEuro 100000;
$rev1 isa revenue, has estimationPrecision 40;
$rev1 isa revenue, has frequency "as needed";
$rev1 isa revenue, has tangible "Yes";
```

TiddlyWiki → TypeDB

Translation
Python Script
.ipynb
(TW→TDB)

Instances

```
else:
    nameRev = node_id
    variable = f"$rev{counters['revenue']}"
    statement = (
        f'insert \n\t{variable} isa {entityType}, has name "{nameRev}";\n'
        f'\t{variable} isa {entityType}, has approximateValueEuro {approximateValueEuro};\n'
        f'\t{variable} isa {entityType}, has estimationPrecision {estimationPrecision};\n'
        f'\t{variable} isa {entityType}, has frequency "{frequency}";\n'
        f'\t{variable} isa {entityType}, has tangible "{tangible}";\n'
        f'\t{variable} isa {entityType}, has referencePeriod "{referencePeriod}";\n'
        f'\t{variable} isa {entityType}, has Comments "{Comments}";\n'
    )
    typedb_statements.append(statement)
    print(statement)
    counters['revenue'] += 1 # Incrementar contador de 'revenue'

with open("typedb_inserts_nodes.gql", "w", encoding="utf-8") as output_file:
    for stmt in typedb_statements:
        output_file.write(stmt + "\n")
```

```
insert
    $rev1 isa revenue, has name "HighResilience";
    $rev1 isa revenue, has approximateValueEuro 100000;
    $rev1 isa revenue, has estimationPrecision 40;
    $rev1 isa revenue, has frequency "as needed";
    $rev1 isa revenue, has tangible "Yes";
    $rev1 isa revenue, has referencePeriod "2025";
    $rev1 isa revenue, has Comments "The system consists of s
```

```
insert
    $rev2 isa revenue, has name "CopropertyObject";
    $rev2 isa revenue, has approximateValueEuro 70000;
    $rev2 isa revenue, has estimationPrecision 40;
    $rev2 isa revenue, has frequency "annually";
    $rev2 isa revenue, has tangible "Yes";
    $rev2 isa revenue, has referencePeriod "2025";
    $rev2 isa revenue, has Comments "Investments to fund the c
```

```
insert
    $serv1 isa service, has name "Thermalism&Tourism";
    $serv1 isa service, has concernedDomain "cultural";
    $serv1 isa service, has tangible "No";
    $serv1 isa service, has Comments "";
```

OUTPUT: .gql
(TypeDB)

Script for TypeDB

Instances

```
Script_TEW-KG > typedb_inserts_nodes.gql
1 insert
2   $infra1 isa infrastructure, has name "GeothermalWell";
3   $infra1 isa infrastructure, has concernedDomain "water management";
4   $infra1 isa infrastructure, has function "water capture process";
5   $infra1 isa infrastructure, has regulated "Yes";
6   $infra1 isa infrastructure, has necessary "Yes";
7   $infra1 isa infrastructure, has constructionDate "1997";
8   $infra1 isa infrastructure, has Comments "";
9
10 insert
11   $cost1 isa cost, has name "InfrastructureManagement";
12   $cost1 isa cost, has approximateValueEuro 100000;
13   $cost1 isa cost, has estimationPrecision 40;
14   $cost1 isa cost, has frequency "irregular";
15   $cost1 isa cost, has planned "Yes";
16   $cost1 isa cost, has referencePeriod "1990-2025";
17   $cost1 isa cost, has Comments "Complete system, including the well, 2 pumps, collector, ...";
18
19 insert
20   $rev1 isa revenue, has name "HighResilience";
21   $rev1 isa revenue, has approximateValueEuro 100000;
22   $rev1 isa revenue, has estimationPrecision 40;
23   $rev1 isa revenue, has frequency "as needed";
24   $rev1 isa revenue, has tangible "Yes";
25   $rev1 isa revenue, has referencePeriod "2025";
26   $rev1 isa revenue, has Comments "The system consists of short, shallow pipelines serving ...";
27
```

```
Query  </> code  chat  ALPHA  Run
1 insert
2   $infra1 isa infrastructure, has name "GeothermalWell";
3   $infra1 isa infrastructure, has concernedDomain "water management";
4   $infra1 isa infrastructure, has function "water capture process";
5   $infra1 isa infrastructure, has regulated "Yes";
6   $infra1 isa infrastructure, has necessary "Yes";
7   $infra1 isa infrastructure, has constructionDate "1997";
8   $infra1 isa infrastructure, has Comments "";
```

IT WORKS!!!

Script for TypeDB

INPUT: .json
(TiddlyWiki)

Relations

```
# insertar una relación entre instancias que ya están dentro de la base de datos
match
  $u0 isa costs, has quantity 4000;
  $u1 isa infrastructure, has name "Balneary heating";
insert
  product (produced: $u0, produces: $u1);
```

```
'tmap.edges': '{"79a55657-2e15-40be-8b01-35adf39a584d":{"to":"782b22bd-13ba-4a69-8fbb-54d5af6c76d5","type":"needs"},"241b3634-7065-4e44-a940-fc4807b
```

Translation
Python Script
.ipynb
(TW→TDB)

```
edges_dict = {}
# Para cada elemento de la lista, tomar el elemento "tmap.edges"
for node in nodesAll:
    nomNode = node["title"]
    edges_json = node.get("tmap.edges", "")

    if edges_json: # Verifica que no esté vacío
        print(f"{nomNode} has edges.")
        edges_data = json.loads(edges_json) # convierte el texto(str) en diccionario de python
        # Crear un subdiccionario de relaciones para cada edge
        edgeRelation = {}
        for edge_id, edge_info in edges_data.items():
            edgeRelation[edge_id] = edge_info
        # Asignar al nombre del nodo (nomNode) el subdiccionario de los edges
        edges_dict[nomNode] = edgeRelation
    else:
        print(f"{nomNode} has NO edges.")

print("Edges Dictionary: ")
print(edges_dict)
```

```
GeothermalWell has edges.
InfrastructureManagement has NO edges.
HighResilience has edges.
CopropertyObject has NO edges.
Thermalism&Tourism has NO edges.
Edges Dictionary:
{'GeothermalWell': {'79a55657-2e15-40be-8b01-35adf39a584d': {'to':
```

Script for TypeDB

INPUT: .json
(TiddlyWiki)

Relations

```
# insertar una relación entre instancias que ya están dentro de la base de datos
match
  $u0 isa costs, has quantity 4000;
  $u1 isa infrastructure, has name "Balneary heating";
insert
  product (produced: $u0, produces: $u1);
```

```
'tmap.edges': '{"79a55657-2e15-40be-8b01-35adf39a584d":{"to":"782b22bd-13ba-4a69-8fbb-54d5af6c76d5","type":"needs"},"241b3634-7065-4e44-a940-fc4807b
```

```
# Create relation structure
#relation_dict={}

for nomNode, edges in edges_dict.items():
    if nomNode == nameInfra:
        for edge_id, edge_info in edges.items():
            edgeType = edge_info.get("type", "")[:-1]
            relation = f"insert\n\t" + edgeType + " (" + edgeType + "ed: " + edge_info.get("to", "") + ", " + edgeType + ": " + variable
            typedb_statements.append(relation)
            print(relation)
counters['infrastructure'] += 1 # Incrementar contador de 'infrastructure'
```

OUTPUT: .gql
(TypeDB)

IT DOESN'T
WORKS
YET!!!

```
insert
  need (needed: 782b22bd-13ba-4a69-8fbb-54d5af6c76d5, need: $infra1);
insert
  provide (provided: b7ab591c-c26d-4155-92cb-cad6a8e8870d, provide: $infra1);
insert
  provide (provided: b76d71b7-6166-4406-8816-d61b0780668b, provide: $infra1);
```

TypeDB

Constraints & Weaknesses

Outputs Errors

ATENCIÓN: UN ERROR EN EL CÓDIGO: EL PROGRAMA DECIDE LAS COLUMNAS QUE MUESTRA EN LA TABLA EN FUNCIÓN DE LOS ATRIBUTOS QUE TIENE LA PRIMERA INSTANCIA. SI LA PRIMERA INSTANCIA TIENE ALGUN ATRIBUTO SIN VALOR (NULL), NO VA A MOSTRAR ESTE ATRIBUTO. Se puede observar en la siguiente figura que el atributo 'type' no aparece porque la instancia "Balneary heating" tiene el atributo 'type' vacío.

</> Query Run

```
1 match
2   $x isa infrastructure;
3 fetch {
4   $x.*
5 };
6
7
8
```

Output log table graph raw

status	name
"working"	"Balneary heating"
"working"	"Pou termal"

ATENCIÓN: no me da los costes más altos, me da las dos instancias que tienen todos los atributos con valores.

```
1 match $x isa costs, has quantity > 200;
2 limit 2;
3 fetch {
4   $x.*
5 };
```

Output log table graph raw

concept	quantity	frequency
"chemical analytics"	"300.0dec"	"year"
"maintenance"	"1000.0dec"	"montly"